

LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Faculty of Physics
AI in Physics Research Group

Mass Reconstruction of Galaxy Clusters Using X-rays and CNNs

Group A

Vyron Arvanitis

Supervisor

Michael Walther

August 7, 2025

Contents

1	Introduction	2
2	Galaxy Clusters	2
3	Convolutional Neural Networks (CNNs) for Image Analysis	3
3.1	Motivation for CNNs and ML	4
4	Methodology	5
4.1	Lab Objectives and Workflow	5
4.2	Data Preparation and Preprocessing	5
4.3	Negative Log-Likelihood Loss	7
4.4	Coverage: Evaluating Predictive Uncertainty	8
4.5	Pull Distribution	8
4.6	Regularization and Normalization Techniques	9
4.7	Hyperparameter Optimization with Optuna	9
4.8	Training Procedure	10
5	Model architecture and results	10
5.1	Model with Optuna Results	11
5.2	Trial and Error model	12
6	Conclusion	14
A	Code Listings	15
A.1	Negative loss likelihood	15

1 Introduction

Galaxy clusters are among the most massive structures in the universe, providing valuable insights into both astrophysics and cosmology. Accurately measuring their masses is essential for studies of large-scale structure, galaxy evolution, and the nature of dark matter and dark energy. X-ray observations offer a powerful way to probe the hot intracluster medium and infer cluster masses.

In this work, we investigate the use of convolutional neural networks (CNNs) for the reconstruction of galaxy cluster masses from X-ray images, with a particular focus on evaluating the effectiveness and uncertainty calibration of different neural network architectures. We present the methodology adopted, discussing the rationale behind our model choices and training strategies. Finally, we compare the results of two high-performing models and critically assess their performance, identifying the most reliable approach for this task.

2 Galaxy Clusters

Galaxy clusters are the largest gravitationally bound structures in the Universe, comprising hundreds to thousands of galaxies, vast amounts of hot intracluster gas that emits X-rays, and of dark matter. Typically, galaxy clusters span between 1 and 5 Mpc in diameter, in comparison individual galaxies (such as the Milky way), are on the order of 10–100 kpc. Their total masses are in the range 10^{14} to $10^{15} M_{\odot}$.

The composition of a galaxy cluster includes three main components. First, the galaxies themselves, which contribute only a small fraction of the total mass. Second, the hot intracluster gas (plasma), which dominates the baryonic mass and is heated to temperatures of 10^7 – 10^8 K, emits strongly in X-rays via thermal bremsstrahlung. Finally, dark matter, which is about about 80% of the total mass, a mysterious form of matter that has so far escaped direct detection with any type of telescope, but makes its presence felt through its gravitational pull on the galaxies and hot gas.

Clusters cannot be made only of galaxies, as the observed gravitational potential (measured via the velocities of galaxies and the temperature distribution of the gas) far exceeds the mass inferred from visible matter alone. This discrepancy highlights the crucial role of both the hot intracluster medium (ICM) and dark matter in cluster dynamics and structure.

In different observable wavelengths, galaxy clusters exhibit different features. In optical light, they appear as dense concentrations of galaxies. In contrast, X-ray observations reveal emission from the hot ICM, outlining the gravitational potential of the cluster. Observatories such as Chandra, XMM-Newton, and eROSITA are sensitive to this emission in the 0.1–10 keV energy range.

The X-ray emission from the ICM is primarily due to thermal bremsstrahlung (free-free emission), which depends on the density and temperature of the gas. The spatial distribution of the X-ray emitting gas is often modeled using the β -model (see [3]):

$$\rho(r) = \rho_0 \left[1 + \left(\frac{r}{r_c} \right)^2 \right]^{-3\beta/2},$$

where ρ_0 is the central gas density, r_c is the core radius, and β is a slope parameter. By projecting this profile along the line of sight, one obtains the X-ray surface brightness profile, which can be fit to observational data.

Under the assumption of an isothermal gas in hydrostatic equilibrium, the total gas mass M_g of a galaxy cluster can be estimated using the β -model as:

$$M_g = \pi^{3/2} \rho_{g0} r_c^3 \frac{\Gamma\left(\frac{3\beta-1}{2}\right)}{\Gamma\left(\frac{3\beta}{2}\right)} \quad \text{for } \beta > 1$$

Alternatively, in terms of the central gas number density n_0 ,

$$M_g = 3.15 \times 10^{12} M_\odot \left(\frac{n_0}{10^{-3} \text{ cm}^{-3}} \right) \left(\frac{r_c}{0.25 \text{ Mpc}} \right)^3 \frac{\Gamma\left(\frac{3\beta-1}{2}\right)}{\Gamma\left(\frac{3\beta}{2}\right)}$$

where:

- n_0 is the central gas number density,
- r_c is the core radius of the cluster,
- β is the slope parameter from the β -model,
- Γ is the gamma function,
- ρ_{g0} is the central gas density.

These relations allow us to estimate the gas and total mass profiles of a cluster from observed X-ray surface brightness and gas temperature, providing key information for understanding the gravitational mass distribution in galaxy clusters.

3 Convolutional Neural Networks (CNNs) for Image Analysis

Convolutional Neural Networks (CNNs) are a class of deep learning models particularly effective for image analysis. In this lab, due to the nature of our data (images of galaxy clusters) we will build and train CNNs. For more theoretical background, see Goodfellow et al. [2] and the TensorFlow CNN tutorial¹.

A standard CNN typically consists of the following components:

- **Input layer:** Accepts the input image data.

¹<https://www.tensorflow.org/tutorials/images/cnn>

- **Convolutional layers (Conv2D):** Apply filters (kernels) to extract spatial features from the input.
- **Activation functions:** Introduce nonlinearity (commonly ReLU).
- **Pooling layers:** Downsample feature maps, typically via max pooling.
- **Dropout layers:** Reduce overfitting by randomly deactivating neurons during training.
- **Fully connected layers:** Integrate features to make final predictions.
- **Output layer:** Produces the classification result.

The core of a CNN is the **convolutional layer**, this layer applies a set of learnable filters (kernels) on the input. Each filter slides across the input image (or previous feature map), performing an element-wise multiplication and summing the result to produce a *feature map*. Important hyperparameters include:

- **Kernel size:** The spatial dimensions of the filter (e.g., 3×3).
- **Filters:** The number of different filters (feature maps) in the layer.
- **Stride:** The step size the filter moves with; higher stride means more downsampling.
- **Padding:** Controls if the input is padded (e.g., with zeros) at the borders.

A key difference between CNNs and fully connected (dense) neural networks is that CNNs exploit *local spatial correlations* in image data through weight sharing. Weight sharing in CNNs means that the same set of filter weights is applied across the entire input image as the filter moves. This enables the network to detect the same pattern regardless of its position in the image. Whereas, on the other hand, dense networks connect every input to every neuron in the next layer and tend to capture more global features. CNNs are therefore more efficient and effective for processing image data. One could also note that if we apply a filter that is the same size as the input image then the convolution operation becomes equivalent to a fully connected (dense) layer therefore CNNs can be thought of as a "generalization" of FNNs.

3.1 Motivation for CNNs and ML

Our current methods for estimating the mass of galaxy clusters from X-ray data rely on physical models that assume the intracluster gas is in hydrostatic equilibrium. While these analytical approaches (such as the β -model) have been very successful, they are based on simplifying assumptions, such as spherical symmetry and – hydrostatic equilibrium, which may not fully capture the complexities of real clusters—especially when clusters are disturbed. As a result, such models may be limited in accuracy or applicability for certain cluster populations.

Machine learning (ML), and Convolutional Neural Networks (CNNs) in particular, offer a complementary approach. CNNs are designed to extract subtle spatial and spectral patterns from images, making them ideally suited for X-ray observations. By training on datasets of simulated clusters, CNNs can learn to recognize features correlated with cluster mass that may not be accessible to traditional analytical methods. Furthermore, ML approaches can generalize to

situations where the underlying physical assumptions are only approximately valid, or where the data is too complex for simple analytic models.

4 Methodology

4.1 Lab Objectives and Workflow

The primary goal of this lab is to develop a machine learning approach for estimating the masses of galaxy clusters directly from X-ray image data. We begin with a simple regression task, using a convolutional neural network (CNN) to predict the cluster mass from the simulated images. As we progressed, we encountered challenges such as overfitting and limited generalization, this motivated us to use various regularization techniques—including dropout, batch normalization, and data augmentation—which we discuss in subsequent sections.

After improving the basic model with regularization, we incorporated additional astrophysical features, such as the cluster redshift, to further enhance the predictive performance. We systematically evaluated each model by training on a split dataset (training, validation, and test sets) and assessed the results through scatter plots of predicted versus true masses. Finally, we extended the network to predict not only the mean mass but also its uncertainty, by modeling the output as a Gaussian distribution and training with a log-likelihood loss.

4.2 Data Preparation and Preprocessing

Data loading. The dataset used in this lab consists of simulated galaxy clusters based on the eROSITA Final Equatorial-Depth Survey (eFEDS [1]). Specifically, the X-ray images and associated catalogues are derived from mock event files and simulations that mimic the expected performance and observational properties of eROSITA prior to its all-sky survey. These simulations were designed to reproduce realistic data, and have been processed with detection and extent likelihood cuts to select a high-quality sample of clusters.

Each cluster is represented by a simulated X-ray image of shape $50 \times 50 \times 10$, where 50×50 are the pixel dimensions and 10 is the number of energy channels (bands) per image. The dataset contains a total of 7,946 such cluster images:

- Number of clusters: $N = 7,946$
- Image shape: $50 \times 50 \times 10$: (height $\times$ width $\times$ energy bands)

For each cluster, these are the available features extracted from the dataset:

- **index** – Unique integer index of the cluster
- **SRC_ID_IN** – Source identification number (input)
- **RA, DEC** – Right Ascension and Declination (sky coordinates)
- **z** – Redshift
- **kT** – Plasma temperature (keV)

- d_L – Luminosity distance
- $L_{X,\text{soft}}, L_{X,\text{soft}}^{2R_{500}}$ – X-ray luminosity in the soft band (total and within $2R_{500}$)
- $F_{X,\text{soft}}, F_{X,\text{soft}}^{2R_{500}}$ – X-ray flux in the soft band (total and within $2R_{500}$)
- M_{500c} – Cluster mass within R_{500c}
- $R_{500c}^{\text{arcmin}}, R_{500c}^{\text{kpc}}$ – Cluster radius R_{500c} in arcminutes and kiloparsecs
- **pid** – Process ID
- $M_{\text{vir}}, R_{\text{vir}}$ – Virial mass and virial radius
- X_{off} – Offset of cluster center
- b/a_{500c} – Minor-to-major axis ratio within R_{500c}
- **ID_SRC_OUT** – Source ID (output)
- $\text{RA}_{\text{eSASS}}, \text{DEC}_{\text{eSASS}}$ – Coordinates from eSASS pipeline
- $\text{RADEC}_{\text{ERR}}$ – Uncertainty in sky position
- **EXT, EXT_ERR, EXT_LIKE** – Source extent, uncertainty, and likelihood
- $\text{ML}_{\text{RATE}}, \text{ML}_{\text{RATE,ERR}}$ – Maximum likelihood count rate and its error
- DET_{LIKE} – Detection likelihood
- $\text{ML}_{\text{BKG}}, \text{ML}_{\text{EXP}}$ – Maximum likelihood background and exposure
- **input_SRC_ID** – Input source identification number
- **Nreal** – Number of real sources associated
- $\text{CLU}_X, \text{CLU}_Y$ – Cluster X and Y pixel coordinates

These features include the cluster’s properties (such as mass, redshift, X-ray luminosity), sky position (RA, DEC) and error measurements

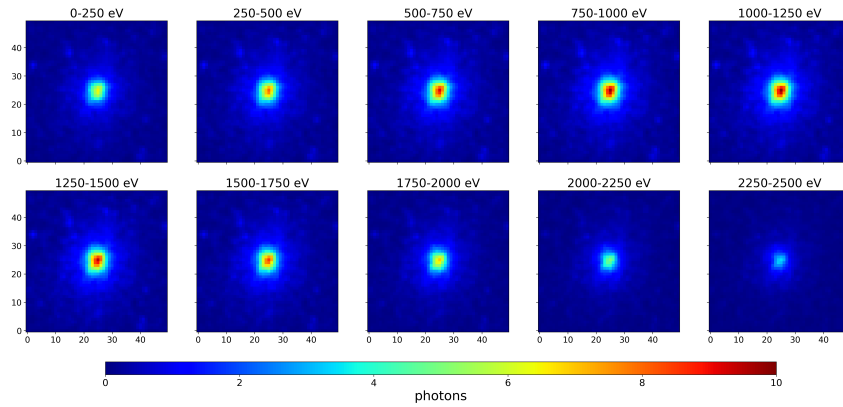


Figure 1: Example of a simulated X-ray image from the eFEDS dataset used in this lab. The image represents the photon energy distribution for a mock galaxy cluster, for several different, energy bands.

Data loading is handled by a dedicated `data_loader` class, which parses the image and catalog files and returns a NumPy array of images and a dictionary of features for each cluster.

Feature normalization. Before training, each feature used (such as cluster mass and redshift) is normalized to have zero mean and unit variance:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

where μ and σ are the mean and standard deviation computed from the training data. This normalization ensures numerical stability – large outliers are normalized and do not have big impact either on the training nor the evaluation of our model. The normalization parameters are stored, enabling inverse transformation (“denormalization”) for interpretation of model predictions. The denormalization was necessary in order to gain interpretable results of the mass, since after normalizing all our results gather around zeron – which makes no physical sense (zero mass?)

Image normalization. To account for variation in overall image intensity, each image is also normalized to zero mean and unit variance, either per-image or per-channel. In this project, per-image normalization is used:

$$I_{\text{norm}} = \frac{I - \langle I \rangle}{\sigma_I}$$

where $\langle I \rangle$ and σ_I are computed over all pixels and channels of the image.

Dataset splitting. The dataset is randomly partitioned into training, validation, and test sets using stratified splits:

- 70% for training
- 15% for validation
- 15% for testing

Splitting is performed using fixed random seeds for reproducibility.

Image augmentation. To improve generalization of our model we increase the size of the training data, via performing image augmentation to the training images. Augmentation operations include random horizontal and vertical flips, small random rotations, random zoom, and random cropping. Further details of Image augmentation will be discussed in the models section [5](#)

4.3 Negative Log-Likelihood Loss

To enable the neural network to not only predict the cluster mass but also to estimate its uncertainty, we adopt a probabilistic approach. We model the predicted mass for each cluster as a Gaussian random variable with mean $\mu = f(x, \theta)$ and variance $\sigma^2 = \sigma(x, \theta)^2$ output by the neural network.

The likelihood for a single true mass value y given input x and network parameters θ is

$$p(y|x, \theta) = \frac{1}{\sqrt{2\pi\sigma(x, \theta)^2}} \exp\left(-\frac{(y - f(x, \theta))^2}{2\sigma(x, \theta)^2}\right). \quad (1)$$

During training, we wish to maximize the likelihood – equivalently, **minimize** the negative log-likelihood over the training set. The negative log-likelihood (NLL) for a single data point is:

$$\mathcal{L}_{\text{NLL}} = -\log p(y|x, \theta) = \frac{1}{2} \log\left(2\pi\sigma(x, \theta)^2\right) + \frac{(y - f(x, \theta))^2}{2\sigma(x, \theta)^2} \quad (2)$$

In our implementation, the neural network is designed so that the output for each sample consists of two values: the predicted mean μ (for the mass) and the predicted variance σ^2 (for the uncertainty). The loss function used in training is the average NLL over all samples in a batch:

$$\text{loss}_{\text{NLL}} = \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{2} \log\left(\sigma_i^2 + \epsilon\right) + \frac{(y_i - \mu_i)^2}{2(\sigma_i^2 + \epsilon)} \right] \quad (3)$$

where ϵ is a small constant ("jitter") added for numerical stability (no divisions by 0). An attentive reader could observe that there is a term missing, however the " π " term is omitted as it is just a constant! The code implementation of the NLL can be seen in the [Appendix A.1](#)

4.4 Coverage: Evaluating Predictive Uncertainty

When our neural network predicts both a mean value and an associated standard deviation for the target variable (e.g., cluster mass), it is important to assess whether these predicted uncertainties are calibrated—that is, whether they accurately reflect the true variability in the data.

The metric we use is coverage, which quantifies the fraction of true target values that fall within a predicted confidence interval. We use a symmetric interval defined by the predicted mean μ and predicted standard deviation σ (i.e., $[\mu - \sigma, \mu + \sigma]$), the coverage is given by:

$$\text{Coverage} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\left(y_i^{\text{true}} \in [\mu_i - \sigma_i, \mu_i + \sigma_i]\right), \quad (4)$$

where $\mathbb{I}$ is the indicator function, N is the number of test examples, y_i^{true} is the true target value, and (μ_i, σ_i) are the predicted mean and standard deviation for the i -th sample.

4.5 Pull Distribution

We use another helpful metric, in order to assess not only the accuracy but also the calibration of our predicted uncertainties, we analyze the distribution of the *pull* values. The pull for each test sample is defined as

$$\text{Pull} = \frac{y_{\text{true}} - \mu_{\text{pred}}}{\sigma_{\text{pred}}}, \quad (5)$$

where y_{true} is the true target value, μ_{pred} is the predicted mean, and σ_{pred} is the predicted standard deviation (uncertainty) for each test point.

Ideally, if the model is both accurate and well-calibrated, the distribution of pull values across the test set should be centered around zero (then we have unbiased predictions) and have a standard deviation close to one.

4.6 Regularization and Normalization Techniques

To achieve better generalization and stable training, our most successful convolutional neural network models employ several normalization and regularization techniques:

- **Batch Normalization:** Batch normalization layers are applied after convolutional and dense layers to normalize intermediate activations. This accelerates and achieves a more stable and faster convergence
- **Dropout:** Dropout is used in the fully connected (dense) layers to prevent overfitting by randomly deactivating a subset of neurons during training.
- **Max Pooling:** Max pooling layers are inserted between convolutional layers to downsample feature maps, reducing the spatial dimensions and encouraging the network to learn translation-invariant features.
- **Padding:** “Same” padding is used in convolutional layers to maintain the spatial resolution of the input throughout the network.

The aforementioned techniques help prevent overfitting, promote generalization, and ensure that our models can extract meaningful patterns from the data without memorizing noise.

4.7 Hyperparameter Optimization with Optuna

To identify the optimal architecture and training configuration for our convolutional neural network, we performed a hyperparameter search using the **Optuna** framework.

In each Optuna trial, a new model configuration is sampled from a predefined search space, trained on the training set, and evaluated on the validation set. The objective metric minimized during optimization was the mean squared error (MSE) on the validation set.

In our study, we employed two different loss functions at different stages of model development. During hyperparameter optimization with Optuna, we minimized the mean squared error (MSE) on the validation set (while having the NLLoss). MSE was chosen for this regression task, as it is easy to interpret and provides a direct measure of prediction accuracy for the target variable.

Thus, while MSE was used for hyperparameter search due to its simplicity and interpretability, the final models were trained and evaluated with the NLL loss to make full use of the model’s capabilities.

The following hyperparameters were tuned in our study:

- **Input Features:** We tested combinations of additional scalar features appended to the image input, including redshift (z), temperature (kT), and soft-band X-ray luminosity (L_{x_soft}).
- **Convolutional Layers:** The number of convolutional layers (2–4), number of filters per layer (16, 32, 64, 128), kernel sizes ($\{10 \times 10, 5 \times 5, 3 \times 3, 2 \times 2\}$), use of max pooling, and application of batch normalization were all varied independently.
- **Fully Connected Layers:** The number of dense (fully connected) layers (1–2), units per layer (64, 128, 256), inclusion of batch normalization and dropout, and dropout rates (0.2–0.5) were explored.
- **Head Layers:** The number of head layers (1–2), units per layer (64, 128, 256), use of batch normalization and dropout, and dropout rates (0.2–0.5).
- **Learning Rate:** The learning rate for the Adam optimizer was tuned on a logarithmic scale in the range $[10^{-4}, 10^{-2}]$.
- **Data Augmentation:** We tested whether to apply random image augmentations during training or not.

At each trial, the model was trained for up to 100 epochs with early stopping –with patience 20 and evaluated on the validation set. Results from all trials were periodically checkpointed and saved for reproducibility and further analysis.

4.8 Training Procedure

For training the final models, we used the Adam optimizer, the models were trained to minimize the negative log-likelihood (NLL) loss function in order to predict both the mean and uncertainty (variance) of the target mass for each galaxy cluster. As a metric for monitoring model performance during training, we tracked the mean squared error (MSE) on the training and validation sets.

We trained all models using a batch size of 64, for up to 100 epochs, and employed early stopping with a patience of 20 epochs.

5 Model architecture and results

Image Augmentation. The image augmentation that we used in the models is the following:

- `RandomFlip("horizontal_and_vertical")`
- `RandomRotation(0.1)`
- `RandomZoom(0.1)`
- `RandomCrop(45, 45)`
- `Resizing(50, 50)`

5.1 Model with Optuna Results

The architecture found by Optuna is summarized in Table 1. Apart from using redshift, the plasma temperature (kT) as an auxiliary input was found to be beneficial to the model by the optuna study. This model achieved a best validation MSE of 0.073 and best negative log-likelihood (NLL) of -0.08136 . The coverage within the predicted 1σ interval was 66.8%. These results are summarized visually in Figures 2 and 3.

Table 1: Summary of the best CNN architecture (Optuna-selected) used for mass regression, including auxiliary features redshift and temperature as inputs.

Layer (type)	Output Shape	Param #
Input (image)	(None, 50, 50, 10)	0
Conv2D (32 filters, 5×5)	(None, 50, 50, 32)	8,032
Conv2D (32 filters, 2×2)	(None, 50, 50, 32)	4,128
Conv2D (128 filters, 2×2)	(None, 50, 50, 128)	16,512
BatchNormalization	(None, 50, 50, 128)	512
Flatten	(None, 320000)	0
Dense (64 units)	(None, 64)	20,480,064
Dropout	(None, 64)	0
Dense (256 units)	(None, 256)	16,640
Dropout	(None, 256)	0
Input (redshift)	(None, 1)	0
Input (kT)	(None, 1)	0
Concatenate	(None, 258)	0
Dense (256 units)	(None, 256)	66,304
Dense (2 units, output)	(None, 2)	514
Lambda (final output)	(None, 2)	0
Total		20,592,706
Trainable params		20,592,450
Non-trainable params		256

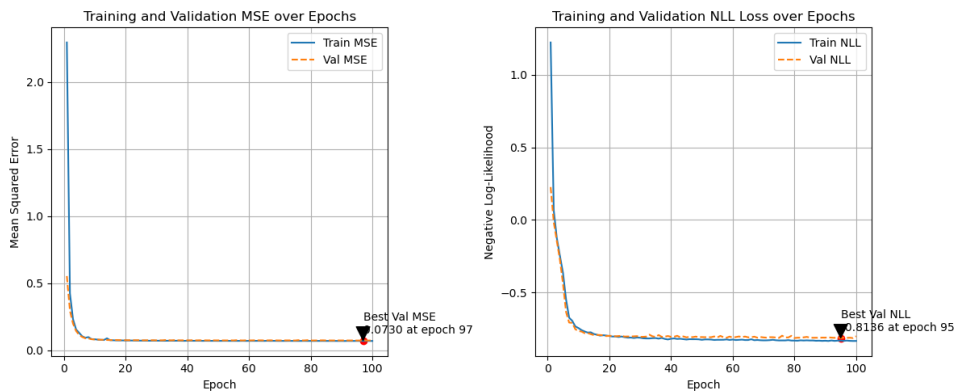


Figure 2: Training curves for MSE and NLL as functions of epoch.

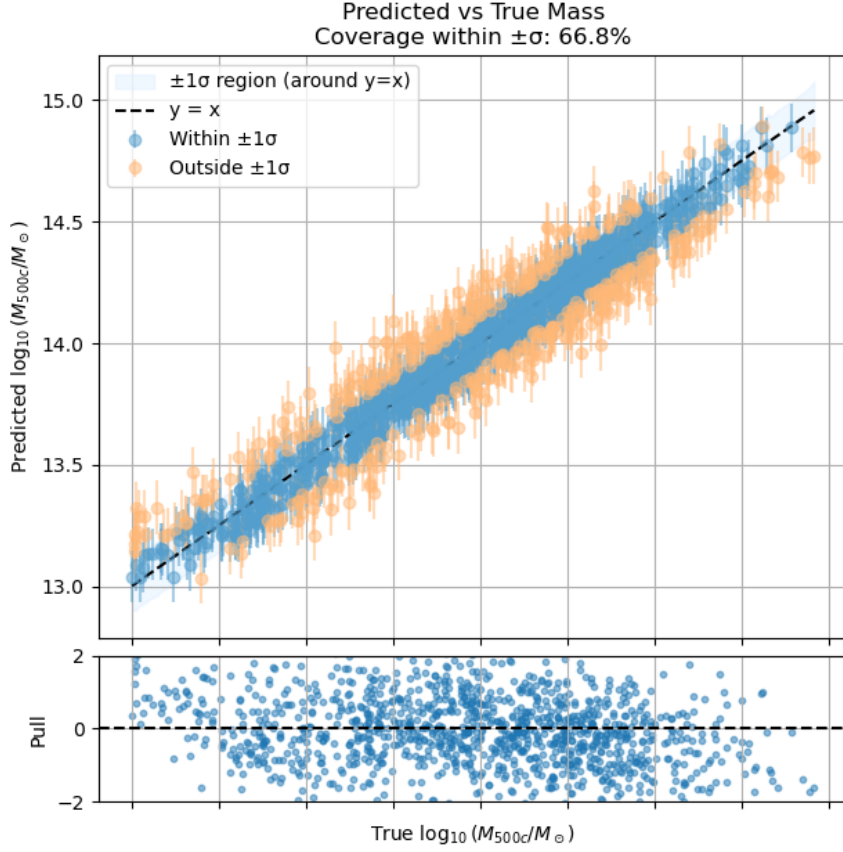


Figure 3: Predicted vs true mass, $\pm 1\sigma$ coverage, and pull distribution.

5.2 Trial and Error model

This model achieved a best validation mean squared error (MSE) of 0.217 and a best negative log-likelihood (NLL) of 0.0490. The corresponding coverage within $\pm 1\sigma$ was 90.3%, indicating improved uncertainty calibration compared to the Optuna-selected model. The key training curves and predictive performance are summarized in Figures 4 and 5.

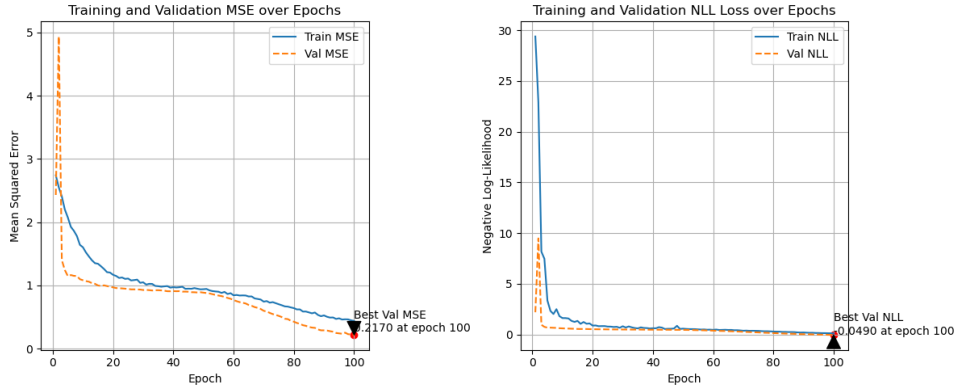
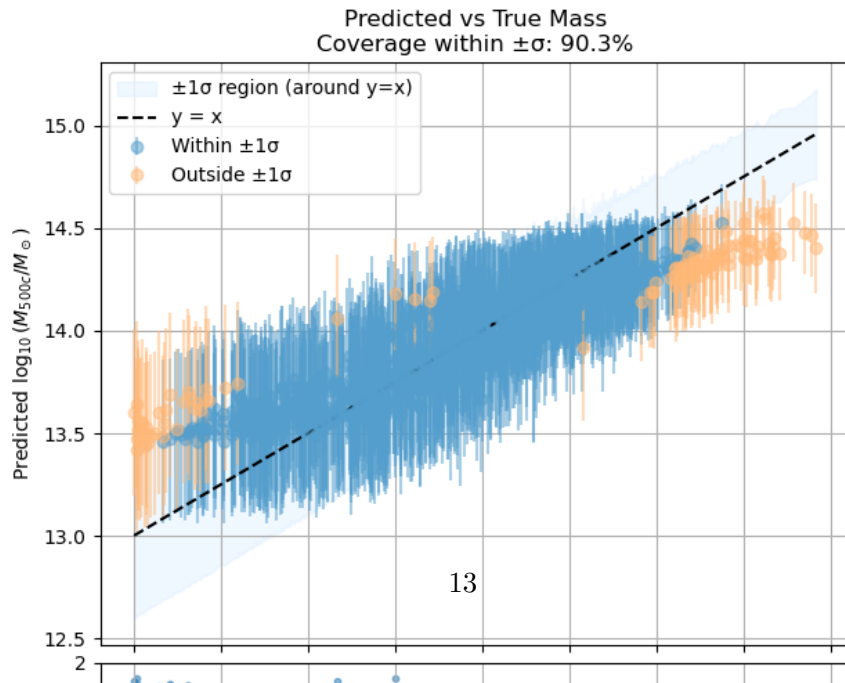


Figure 4: Training curves for MSE and NLL as functions of epoch.

Table 2: Summary of the regularized CNN architecture with extensive use of BatchNorm and Dropout.

Layer (type)	Output Shape	Param #
Input (image)	(None, 50, 50, 10)	0
Conv2D (16 filters, 3×3)	(None, 50, 50, 16)	16,016
BatchNormalization	(None, 50, 50, 16)	64
MaxPooling2D	(None, 25, 25, 16)	0
Conv2D (32 filters, 3×3)	(None, 25, 25, 32)	12,832
BatchNormalization	(None, 25, 25, 32)	128
Conv2D (64 filters, 3×3)	(None, 25, 25, 64)	51,264
BatchNormalization	(None, 25, 25, 64)	256
MaxPooling2D	(None, 12, 12, 64)	0
Conv2D (128 filters, 3×3)	(None, 12, 12, 128)	32,896
BatchNormalization	(None, 12, 12, 128)	512
MaxPooling2D	(None, 6, 6, 128)	0
Flatten	(None, 4608)	0
Dense (64 units)	(None, 64)	294,976
BatchNormalization	(None, 64)	256
Dropout	(None, 64)	0
Dense (32 units)	(None, 32)	2,080
Dropout	(None, 32)	0
Input (redshift)	(None, 1)	0
Input (kT)	(None, 1)	0
Concatenate	(None, 34)	0
Dense (16 units)	(None, 16)	560
BatchNormalization	(None, 16)	64
Dropout	(None, 16)	0
Dense (4 units)	(None, 4)	68
Dropout	(None, 4)	0
Dense (2 units, output)	(None, 2)	10
Lambda (final output)	(None, 2)	0
Total		411,982
Trainable params		411,342
Non-trainable params		640



6 Conclusion

In Table 3, we summarize the final performance metrics for our two best models.

The **Regularized CNN model** achieved the lowest validation MSE and NLL, with a coverage of 66.8%, which is close to the expected 68% for a well-calibrated Gaussian model. This indicates that the model not only provides accurate mass predictions but also delivers reliable uncertainty estimates.

In contrast, the **Trial and Error model** produced a very high coverage of 90.3%. While this might seem impressive, it can be misleading: such high coverage indicates that the model’s uncertainty estimates are very conservative, meaning it predicts error bars that are too wide. As a result, the model may appear to capture more true values within its predicted intervals, but this comes at the cost of less informative, less precise predictions. This interpretation is further supported by its higher validation MSE and NLL, which indicate that the Trial and Error model’s mass predictions are less accurate and its probabilistic predictions less well-calibrated.

Additionally, from the pull distributions, we can further assess the quality of the models. For the Regularized CNN model, the pull values are evenly distributed around zero, indicating well-calibrated uncertainties and no systematic bias in the predictions. In contrast, the pull plot for the Trial and Error model exhibits an **S-shaped** pattern, suggesting that its uncertainties are not well calibrated and may exhibit systematic deviations. This further supports our belief that, despite its higher coverage, the Trial and Error model does not provide reliable uncertainty estimates.

Therefore, we conclude that the Regularized CNN model provides a better balance between prediction accuracy and uncertainty calibration for this task.

Table 3: Comparison of final performance metrics for the two best models.

Model	Best Val NLL	Best Val MSE	Coverage (%)
Regularized CNN mode	−0.08136	0.073	66.8
Trial and Erro model	0.0490	0.217	90.3

Potential Improvements: The aforementioned work is by no means perfect and can be improved in several ways. First, repeating the hyperparameter optimization with Optuna using the negative log-likelihood (NLL) as the objective would directly target better uncertainty calibration. Additionally, experimenting with more advanced architectures—such as hybrid models that combine Convolutional Neural Networks (CNNs) with Transformers, or using Vision Transformers (ViTs) and attention-based mechanisms—could further enhance performance on image data.

Another potential direction involves a more detailed investigation of the auxiliary input features. In this study, we included the full range of redshift and plasma temperature (kT) values as input features. Future work could analyze the distributions of these features in more detail, potentially identifying specific ranges where the model’s performance is stronger or where the data is more reliable.

Appendix

Note: LLMs like DeepSeek and ChatGPT were used for spell check and debugging of the code. They also assisted with creating docstrings.

A Code Listings

A.1 Negative loss likelihood

```
def loss_nll(y_true: tf.Tensor, y_pred: tf.Tensor) -> tf.Tensor:
    """Computes the negative log-likelihood (NLL)

    Parameters
    -----
    y_true : tf.Tensor
        True target values. Shape: (batch_size,) or (batch_size, 1).
    y_pred : tf.Tensor
        Predicted mean and variance for each input, as output by the
        network.
        Shape: (batch_size, 2), where y_pred[:, 0] = mean, y_pred[:,
        1] = variance.

    Returns
    -----
    tf.Tensor
        Scalar tensor representing the mean NLL loss over the batch.
    """
    y_true = tf.squeeze(y_true) # shape (batch_size,)
    mu = y_pred[:, 0]           # predicted means
    var = y_pred[:, 1]          # predicted variances (must be
                                positive)
    jitter = 1e-8               # numerical stability
    nll = 0.5 * tf.math.log(var + jitter) + 0.5 * tf.square(y_true -
        mu) / (var + jitter)
    return tf.reduce_mean(nll)
```

References

- [1] eFEDS catalog v3.3 (mock and observed catalogs). https://erosita.mpe.mpg.de/edr/eROSITAObservations/Catalogues/liuT/eFEDS_catalog_V3.3.html. Accessed on August 7, 2025.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] Craig L. Sarazin. X-ray emission from clusters of galaxies (ned/ipac). <https://ned.ipac.caltech.edu/level5/March02/Sarazin/frames.html>, 2002. Accessed: August 7, 2025.